

Password-Authenticated Key Exchange Protocols

Pratheek Dhananjaya and Fatemeh Jafarian Dehkordi

1 Introduction

The core challenge in security is the inherent discrepancy between high-entropy cryptographic keys and low-entropy, human-memorizable passwords. Traditionally, protocols like Transport Layer Security (TLS) secure data in transit but still require the client to transmit the plaintext password (or a hash) to the server for validation. This architecture relies on absolute server trust and is vulnerable to two major threat models. First, an active Man-in-the-Middle (MitM) adversary can spoof the server to capture the credentials in transit. Second, and more critically, if an adversary records a standard authentication handshake, they can use that data as a local "verification oracle." This allows them to execute massive offline dictionary attacks, testing millions of password guesses per second on their own hardware without triggering server-side rate limits. Furthermore, this model forces the server to store password-equivalent data, creating a single point of failure if the backend database is breached.

Password-Authenticated Key Exchange (PAKE) protocols [1] were developed to iteratively eliminate these vulnerabilities. A PAKE redesigns the authentication architecture, allowing a client and server to mutually authenticate and subsequently derive a high-entropy cryptographic session key using only a shared, weak password as the root of trust. Most importantly, a PAKE protocol achieves this without ever transmitting the password across the network.

The defining cryptographic guarantee of a PAKE is its resilience against offline dictionary attacks. The mathematical structures of the exchange ensure that the data transmitted over the wire provides zero functional information to an eavesdropper. An attacker cannot use the intercepted network traffic to locally verify whether a candidate password guess is correct. By severing the reliance on vulnerable transport tunnels and eliminating the local verification oracle, PAKE protocols force attackers to rely on online guessing, which is easily thwarted by standard server-side security policies.

2 Relationships to Other Cryptographic Topics

Password-Authenticated Key Exchange protocols naturally intersect several foundational cryptographic primitives, acting as an authenticated evolution of standard key agreement mechanisms. Traditional protocols, such as the Diffie-Hellman key exchange, successfully establish a shared secret over an insecure channel but lack mutual authentication, making them completely vulnerable to active Man-in-the-Middle (MitM) attacks. To authenticate these exchanges, traditional architectures rely on Public Key Infrastructure (PKI), necessitating digital certificates, Certificate Authorities (CAs), and trust chains. A PAKE removes the need for PKI. By injecting the shared low-entropy password directly into the algebraic structure of the key exchange, either symmetrically as an encryption key (as in EKE) or as a blinding factor (as in SRP), PAKEs achieve mutual authentication in a decentralized, peer-to-peer context without relying on external trust anchors.

Furthermore, modern asymmetric PAKEs are linked to the study of Zero-Knowledge Proofs (ZKPs). While early protocols like EKE rely on symmetric cryptographic architecture, asymmetric protocols utilize zero-knowledge principles during the authentication phase. In SRP, the client must mathematically prove possession of the private secret corresponding to the public verifier stored. The protocol's challenge-response mechanism ensures that the transcript of this interaction yields zero functional knowledge to the verifier (the server) regarding the actual password. This integration of zero-knowledge

characteristics guarantees that even a compromised server cannot utilize its stored data to successfully execute a login sequence.

Finally, the efficacy of PAKE protocols is dependent on advancements in Key Derivation Functions (KDFs) and memory-hard algorithms, such as Argon2. While an asymmetric PAKE secures the network transit layer and protects the server from immediate impersonation attacks, the architecture remains theoretically vulnerable to pre-computed offline dictionary attacks if the server’s verifier database is breached. To mitigate this, the initialization and registration phases of a PAKE can integrate memory-hard functions to derive the intermediate secrets. By utilizing algorithms like Argon2, which intentionally bottleneck computational resources via high memory bandwidth requirements, the protocol equalizes the computational cost for legitimate users and attackers utilizing Application-Specific Integrated Circuits (ASICs). This drastically reduces the economic feasibility of massive offline dictionary attacks against stolen verifier databases, making Argon2 and similar KDFs as critical complementary technologies to the PAKE architecture.

3 EKE: Encrypted Key Exchange Protocol

Prior to the early 1990s, the cryptographic community operated under the assumption that establishing a secure, mutually authenticated channel over a fully active, adversarial network required pre-existing high-entropy secrets, such as long cryptographic keys or Public Key Infrastructure (PKI) certificates. It was accepted that using a low-entropy, human-memorizable password to bootstrap a secure key exchange would eventually expose the system to offline dictionary attacks. This assumption was challenged by the introduction of the Encrypted Key Exchange (EKE) protocol [17]. EKE established the foundation for the entire field of Password-Authenticated Key Exchange (PAKE) by presenting the first Type-I symmetric architecture, demonstrating that a weak shared secret could successfully shield a robust public-key exchange.

3.1 The Mathematical Protocol Flow

The core innovation of EKE is the integration of a symmetric encryption cipher into an asymmetric public-key exchange algorithm, most commonly utilizing the Diffie-Hellman (DH) protocol. The architecture assumes that both the client (Alice) and the server share a prior symmetric secret, namely the plaintext password P . Furthermore, both parties agree upon a set of public domain parameters: a large prime modulus N , a generator g of a multiplicative group, and a symmetric encryption algorithm E . The protocol executes in the following mathematical sequence:

1. **Initialization and Encryption:** Alice generates a random, ephemeral private exponent a . She computes her standard Diffie-Hellman public key as $g^a \bmod N$. Instead of transmitting this public key in the clear, which would expose the exchange to active Man-in-the-Middle manipulation, Alice uses the shared password P as a symmetric encryption key. She computes the ciphertext $E_P(g^a \bmod N)$ and transmits it to the server.
2. **Server Decryption and Response:** The server intercepts the ciphertext. Because EKE is a symmetric protocol, the server retrieves Alice’s plaintext password P from its local database and decrypts the message to recover Alice’s public key g^a . The server then generates its own random private exponent b and computes its public key $g^b \bmod N$. The server then computes the master session key $K = (g^a)^b \bmod N$ and encrypts its public key using the shared password, resulting in $E_P(g^b \bmod N)$, and transmits it back to Alice.

3. **Client Key Derivation:** Alice receives $E_P(g^b \bmod N)$ and decrypts it using her locally inputted password P to extract the server's public key g^b . Alice then derives the identical master session key $K = (g^b)^a \bmod N$.
4. **Mutual Authentication:** To confirm that both parties have successfully derived the same key K without transmitting it, they execute a challenge-response utilizing a secondary symmetric cipher and random nonces. Alice generates a random nonce c , encrypts it with the newly derived session key K , and sends $E_K(c)$ to the server. The server decrypts c , generates its own nonce d , and sends back $E_K(c, d)$ to prove it successfully decrypted Alice's challenge. Finally, Alice sends $E_K(d)$ to complete the mutual authentication.

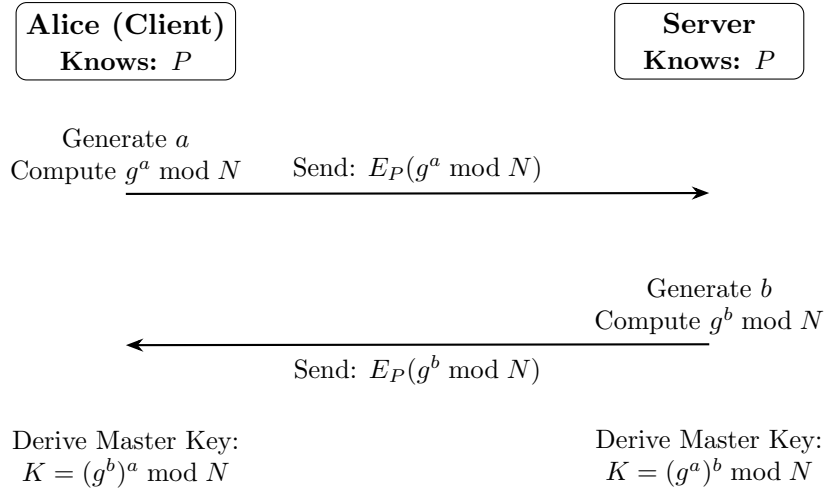


Figure 1: The Encrypted Key Exchange (EKE) protocol flow. The shared password P is used as a symmetric encryption key to shield the Diffie-Hellman public keys from active and passive adversaries.

3.2 Cryptographic Security Guarantee

The brilliance of EKE lies in its ability to neutralize the offline dictionary attack. If a passive adversary intercepts the initial transmission $E_P(g^a)$, their standard operating procedure is to iterate through a dictionary of candidate passwords. The adversary selects a guess, P' , and attempts to decrypt the intercepted ciphertext, computing $D_{P'}(E_P(g^a)) = X$.

In traditional encryption schemes, the plaintext contains known formatting, packet headers, or structural redundancy. An adversary uses this redundancy as a "verification oracle." If the decrypted string X matches the expected format, the guess P' is confirmed correct. EKE eliminates this oracle. Diffie-Hellman public keys are elements of a finite mathematical group and appear as uniformly distributed random strings. When the adversary decrypts the message with an incorrect password P' , the resulting string X is a different random element within that same group. The adversary possesses no local mathematical mechanism to determine if X is Alice's legitimate public key or the output of an incorrect decryption. Consequently, the intercepted network traffic provides zero functional assistance, forcing the adversary to abandon offline pre-computation in favor of highly detectable, rate-limited online guessing.

3.3 Architectural Limitations and the Backend Flaw

While EKE secures the network transit layer against both active MitM and passive offline attacks, its symmetric nature introduces a vulnerability at the backend storage layer. For

the server to participate in the EKE protocol, it must possess the exact same symmetric key as the client. This mandates that the server maintains a database containing the plaintext passwords of every registered user, or at best, an encrypted database where the decryption keys reside on the same host.

This requirement constitutes a severe single point of failure. The threat model of EKE is limited to network-level adversaries. If an adversary shifts their focus from the network to the application layer and successfully breaches the server host via SQL injection, unpatched software vulnerabilities, or compromised administrative credentials, they obtain the plaintext equivalent of the database. With access to this symmetric data, the adversary can immediately and seamlessly impersonate any user on the system without needing to crack any hashes or execute dictionary attacks. This vulnerability underscored the necessity for an evolutionary leap in PAKE design: the development of asymmetric architectures capable of protecting the database itself.

4 SRP: Secure Remote Password Protocol

The vulnerability of symmetric PAKE architectures, specifically the requirement that the server store plaintext-equivalent authentication material, necessitated a paradigm shift. Secure Remote Password (SRP) protocol [18] was introduced, pioneering the implementation of an Asymmetric PAKE (frequently classified as a Type-V PAKE). The objective of SRP was to preserve the security demonstrated by EKE while ensuring that a complete compromise of the server’s backend database would not yield data sufficient to initiate an impersonation attack.

4.1 Registration and the Verifier Construct

SRP achieves this asymmetric security profile by forcing the client to transform the plaintext password into a non-invertible mathematical construct prior to establishing a server record. This transformation removes the symmetric constraint.

During the initialization and registration phase, the client selects a password P . The client’s device locally generates a random cryptographic salt s . The salt and password are concatenated and processed through a one-way cryptographic hash function H to derive a private intermediate secret x :

$$x = H(s, P)$$

The asymmetric property occurs when the client utilizes this private secret x as an exponent within a discrete logarithm group (defined by a large safe prime N and a generator g) to derive a public Verifier v :

$$v = g^x \bmod N$$

The client transmits only the user identifier I , the salt s , and the Verifier v to the server. The server stores these values in its database. Because the derivation of v utilizes modular exponentiation, an adversary (or the server itself) attempting to reverse the equation to discover x (and subsequently P) must solve the discrete logarithm problem, which is computationally intractable for adequately sized parameters. In addition, the server never has the plaintext password or any symmetric equivalent.

4.2 The Authentication Exchange

The authentication phase of SRP operates as a zero-knowledge challenge-response mechanism, allowing both parties to mutually derive a strong session key without exposing their respective secrets. The protocol executes as follows:

1. **Initialization:** The client sends their identifier I to the server. The server retrieves the corresponding salt s and Verifier v from its database, and transmits the salt s back to the client.

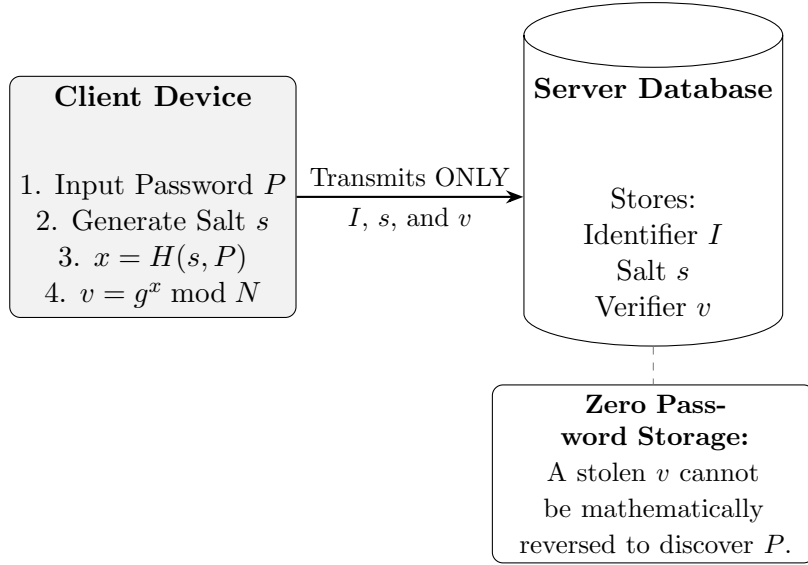


Figure 2: The SRP Registration Phase. The symmetric constraint is severed by forcing the client to utilize a one-way mathematical function prior to server storage, neutralizing the threat of an immediate backend compromise.

2. **Ephemeral Key Generation:** The client uses the received salt s and their locally inputted password P to recalculate the private secret $x = H(s, P)$. The client generates a random ephemeral private key a and computes the public value $A = g^a \bmod N$, transmitting A to the server.

The server generates its own random ephemeral private key b . To cryptographically bind the exchange to the user's password, the server computes a "blinded" public value $B = kv + g^b \bmod N$ (where k is a multiplier parameter derived from hashing N and g). The server transmits B to the client.

3. **Session Key Derivation:** Both parties independently compute a parameter $u = H(A, B)$.
 - The client derives the master session secret S by computing $S = (B - kg^x)^{(a+ux)} \bmod N$.
 - The server derives the identical master session secret S by computing $S = (Av^u)^b \bmod N$.
4. **Verification:** Both parties hash the secret S to yield the final session key $K = H(S)$. They then exchange cryptographic proofs (typically hashes of A , B , and K) to mutually verify that they have arrived at the exact same key, completing the authentication without ever transmitting P , x , or v during the active exchange.

4.3 Security Guarantees and Remaining Vulnerabilities

The mathematical architecture of SRP neutralizes the single point of failure that affected EKE. If an adversary breaches the SRP server and exfiltrates the entire database, they acquire only the salts (s) and the verifiers (v). The SRP protocol demands knowledge of the private secret x to initiate an authentication request as a client. Because the stolen verifier v is non-invertible, the adversary cannot use the stolen database to impersonate users; the verifier can only be used by a server to validate a legitimate attempt.

However, while SRP secures the database against immediate impersonation, it leaves one residual theoretical vulnerability. The stolen verifier v acts as a localized verification

oracle. An adversary possessing the compromised database can mount an offline dictionary attack by guessing a password P' , computing $x' = H(s, P')$, deriving $v' = g^{x'} \bmod N$, and checking if v' matches the stolen verifier v . If the user selected a weak password, a well-resourced attacker utilizing pre-computed dictionary tables (such as rainbow tables) can eventually crack it. This limitation dictates the boundary of Type-V Asymmetric PAKEs, establishing the exact threat model and pre-computation offline attacks against compromised databases, which necessitate the modern transition to Strong Asymmetric PAKEs (saPAKEs) like OPAQUE.

5 OPAQUE: A Strong Asymmetric PAKE

The previous sections traced a natural progression in PAKE design. EKE showed that a low-entropy password can authenticate a strong session key, but it still required symmetric password storage. SRP moved to asymmetric storage by having the server store a verifier, for example $V = g^{H(pw\|salt)}$, rather than the password itself. However, this verifier remains a deterministic function of values that can eventually become available to an attacker, leaving SRP vulnerable to pre-computation attacks. OPAQUE, proposed by Jarecki, Krawczyk, and Xu at EUROCRYPT 2018 [2], addresses this gap.

5.1 The Pre-Computation Attack

Per-user random salts defeat *universal* pre-computation: an attacker cannot maintain a single rainbow table that works against every server, since each server uses fresh salts. They do not, however, defeat a more patient and targeted adversary.

In SRP and in the classical aPAKE protocols that preceded OPAQUE, the salt is transmitted to the client during login; it has to be, since the client needs it to derive the verifier. An attacker who passively observes one legitimate login transcript for user U therefore learns $salt_U$. With $salt_U$ in hand, the attacker can pre-compute, long before any breach occurs,

$$T_U = \{(pw_i, g^{H(pw_i\|salt_U)}) : pw_i \in \mathcal{D}\}.$$

When the verifier file eventually leaks, recovering pw_U becomes an $O(1)$ table lookup. The dictionary work was paid in advance; the breach itself does none of it.

The general lesson is that any value visible in a transcript can be used as an input to offline pre-computation. To prevent this, the password-derived value stored by the server must depend on a secret that never appears on the wire: a server-held value that the attacker only learns at the moment of compromise. The Wang-Li-Jiang survey captures this distinction through resistance criterion S6-9 [1]; SRP, PAK-Z, and SPAKE2+ fail this criterion, while OPAQUE satisfies it.

5.2 The Core Idea

OPAQUE changes both what the server stores and how the password-derived value is obtained. Instead of storing a verifier that is publicly computable from the password and salt, the server stores a per-user OPRF key k_U and an encrypted envelope. During login, the client and server run an OPRF so that only the client learns

$$rwd = F_{k_U}(pw_U).$$

The server contributes the secret key k_U to this computation, but it never learns either pw_U or rwd . Three properties follow from this design:

- Pre-computation becomes useless: the attacker cannot evaluate $F_{k_U}(\cdot)$ before learning k_U , so a useful table cannot be built before the breach.
- The server never sees pw , in any phase: not at registration, not at login.

- The construction admits a UC-framework security proof; we discuss this in §5.7.

The cryptographic tool that makes this possible is the oblivious pseudorandom function.

5.3 The Oblivious Pseudorandom Function

An oblivious pseudorandom function (OPRF) is a two-party protocol that computes $F_k(x)$ between a client holding input x and a server holding key k . The client learns $F_k(x)$ but nothing about k ; the server learns nothing about x , including whether two OPRF queries used the same input. For a fresh key k , the function output is indistinguishable from random to anyone who does not know k .

The OPRF construction used in OPAQUE, called 2HashDH [6], works in a prime-order group $\mathbb{G} = \langle g \rangle$. Let $H_1 : \{0, 1\}^* \rightarrow \mathbb{G}$ and $H_2 : \{0, 1\}^* \rightarrow \{0, 1\}^\ell$ be hash functions. The client samples a fresh blinding factor $r \in_R \mathbb{Z}_q^*$ and sends

$$a = H_1(x)^r$$

to the server. The server computes $b = a^k = H_1(x)^{rk}$ and returns it. The client unblinds by computing $b^{1/r} = H_1(x)^k$ and outputs

$$F_k(x) = H_2(x, H_1(x)^k).$$

The blinding factor hides x from the server. The response b binds the computation to the server's key k , but the client can only unblind the value for its own input. Thus, only the client obtains the OPRF output.

5.4 Registration

Registration runs once over an authenticated channel and creates the password file that the server stores for future logins. Crucially, this setup allows the server to store the envelope E_U without learning pw or the long-term keys protected by the envelope.

1. The server samples a fresh OPRF key k_U for the user.
2. The client and server execute the OPRF with the client's input pw and the server's key k_U . The client obtains $pwd = F_{k_U}(pw)$, while the server learns nothing about pw or pwd .
3. The server provides its static public key P_S to the client.
4. The client generates a long-term key pair (p_U, P_U) with $P_U = g^{p_U}$ and constructs the envelope

$$E_U = \text{AuthEnc}_{pwd}(p_U, P_U, P_S).$$

5. The client uploads (E_U, P_U) to the server and erases pw , pwd , and p_U from local memory.

The OPAQUE paper presents registration abstractly as an authenticated setup phase; the above description is the natural client-side version, where the server never obtains the password or the client's long-term secret key. The authenticated-encryption layer protecting the envelope must be random-key robust, so that a wrong password does not decrypt the envelope into a meaningful but incorrect key. The paper notes that this can be achieved, for example, by using an encrypt-then-MAC construction.

The server stores $file_U = (k_U, E_U, P_U)$, while its own static key pair remains part of its long-term state. A breached server gives the attacker k_U and E_U , but decrypting E_U still requires pwd , which in turn requires the correct password. Therefore, after compromise, the attacker can run a dictionary attack, but the work begins only at the time of compromise rather than being precomputed.

5.5 Login

At a high level, OPAQUE login has two components. First, an OPRF execution lets the client recover a hardened credential $pwd = F_{k_U}(pw)$. Second, an authenticated Diffie-Hellman exchange uses the long-term keys protected by that credential to establish and confirm a session key. In the concrete protocol from the OPAQUE paper, these two components are interleaved: the OPRF messages are piggy-backed on the first two authenticated-DH messages, so the protocol still uses three messages overall.

The client begins login holding only pw . The server holds the password file $file_U = (k_U, E_U, P_U)$, where E_U is an authenticated-encryption envelope constructed during registration. When decrypted with pwd , this envelope recovers the client’s long-term secret key p_U , the corresponding public key P_U , and the server’s static public key P_S . Figure 3 sketches the message flow, including the key-confirmation tags.

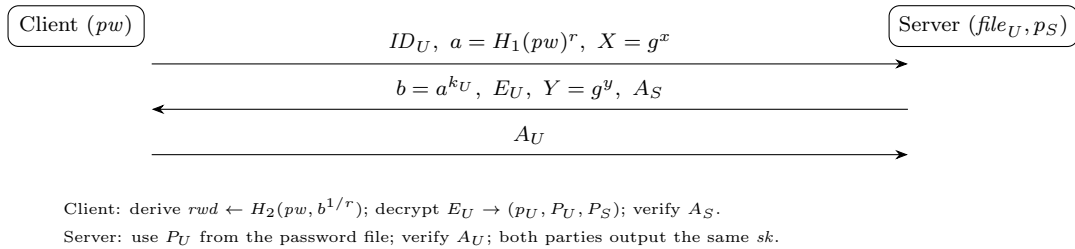


Figure 3: The OPAQUE login protocol. The OPRF computation is interleaved with an authenticated Diffie-Hellman exchange and explicit key confirmation.

Here A_S and A_U are the server and client authentication tags, respectively, derived from the authenticated key-exchange value and used for explicit key confirmation. The authenticated-DH stage is shown abstractly in the figure; we expand on it in §5.6.

5.6 The Authenticated Diffie-Hellman Stage

In our presentation, this step was collapsed into a single arrow labeled “authenticated DH using static and ephemeral keys.” We expand it here, first using a 3DH-style view because it makes the role of the static and ephemeral keys easier to see. This is an explanatory view; the original OPAQUE paper instantiates the AKE component with HMQV.

After the OPRF phase, both parties hold the relevant keys. The client has (p_U, P_U) recovered from E_U , the server's public key P_S from the same envelope, and a fresh ephemeral key pair (x, X) with $X = g^x$. The server has (p_S, P_S) , the client's public key P_U from the password file, and a fresh ephemeral key pair (y, Y) with $Y = g^y$. In a 3DH-style formulation, both parties compute

$$\begin{aligned} \text{DH}_1 &= X^{p_S} = (P_S)^x = g^{x \cdot p_S}, \\ \text{DH}_2 &= (P_U)^y = Y^{p_U} = g^{y \cdot p_U}, \\ \text{DH}_3 &= X^y = Y^x = g^{xy}, \end{aligned}$$

and derive

$$sk = \text{KDF}(\text{DH}_1 \parallel \text{DH}_2 \parallel \text{DH}_3 \parallel \text{transcript}).$$

Each term has a distinct role. DH_1 binds the session to the server’s long-term identity, since only an entity holding p_S can compute it. DH_2 binds the session to the client’s long-term identity, since only a client that correctly decrypted E_U can recover p_U . DH_3 provides forward secrecy: after x and y are erased, later compromise of the static keys does not reveal past session keys. If the client enters the wrong password, it derives the wrong rwd , fails to recover the correct long-term keys from E_U , and obtains a session

key that does not match the server’s. The mismatch is detected through the explicit authentication tags.

The HMQV instantiation used in the original paper expresses the same authenticated-DH goal more compactly. With $d = H(X, ID_S)$ and $e = H(Y, ID_U)$, both parties compute

$$\sigma = (Y \cdot (P_S)^e)^{x+d \cdot p_U} = (X \cdot (P_U)^d)^{y+e \cdot p_S} = g^{(x+d \cdot p_U)(y+e \cdot p_S)},$$

and derive the session key as $sk = \text{KDF}(\sigma, \text{transcript})$. The term $x + d \cdot p_U$ acts as an implicit authentication of the client’s ephemeral value X : only a party that knows p_U can produce the corresponding contribution. This makes HMQV efficient, but its proof is more delicate than the 3DH-style explanation above.

OPAQUE’s modularity is important here. The protocol is not tied to a single AKE: the original paper illustrates the construction using HMQV, but the same framework can use any AKE that satisfies the required properties, including Key Compromise Impersonation (KCI)¹ resistance and full forward secrecy. The paper also notes that a similar result can be argued using SIGMA, although SIGMA-based instantiations may require additional operations such as signatures.

This modularity is not limited to the AKE layer. The OPRF removes the pre-computation problem by deriving a server-secret-dependent value $pwd = F_{k_U}(pw)$, but the client can further harden this value before using it as the envelope key. For example, the paper describes replacing pwd with $H^n(F_{k_U}(pw))$; in practice, one can use password-based key-derivation functions such as PBKDF2, bcrypt, scrypt, or memory-hard functions such as Argon2. This increases the cost of post-compromise offline guessing and slows online guessing, while still keeping the password hidden from the server.

5.7 Security and Costs

Security guarantees. OPAQUE is a strong aPAKE: a UC-secure aPAKE that provably resists pre-computation attacks. The proof in [2] relies on the One-More Diffie-Hellman assumption in the random oracle model, the security of the DH-OPRF, an AKE satisfying the required KCI-resistant functionality, and the robustness of the authenticated encryption protecting the envelope. Before server compromise, the adversary cannot evaluate F_{k_U} on password guesses without interacting with the server. After compromise, the attacker can run an offline dictionary attack using the leaked k_U and E_U , but that work starts at compromise rather than being pre-computed in advance. The construction also inherits forward secrecy from the ephemeral DH stage and password hiding from the OPRF: the server never sees pwd , even during a successful login.

Costs. Relative to a plain Diffie-Hellman handshake, OPAQUE adds one OPRF execution and one envelope decryption on the client side. In the instantiation analyzed in the paper, the protocol requires about three exponentiations for the server and four for the client, compared with two per party for a plain DH-style exchange. The protocol completes in three messages with mutual explicit authentication. The server stores one additional per-user secret, k_U , and a constant-size envelope, E_U .

The main deployment cost is that OPAQUE requires changes on both the client and server sides. Password-over-TLS, by contrast, works with any HTTPS-capable client because password handling is independent of the cryptographic protocol.

¹KCI resistance ensures that an attacker who has stolen a party’s long-term private key can impersonate that party to others, but cannot use the stolen key to impersonate other parties back to the victim. The asymmetry matters because it limits the blast radius of a single key compromise.

6 Synthesis and Open Problems

6.1 Why These Three Protocols?

The Wang-Li-Jiang survey reviews 71 PAKE protocols organized into seven design strategies. Our selection follows three criteria: the protocols should cover different design strategies, trace the historical and conceptual development of PAKE, and expose the main security-usability-efficiency trade-offs identified in the survey.

EKE (1992) is the natural starting point. As the first PAKE, it introduced the basic idea that a weak password can authenticate a strong key exchange. The survey’s historical figure traces several later protocols back to EKE, including KHAPE, aEKE-HMQV, and OPAQUE. EKE represents the Type-I strategy, where the password is used as an encryption key, and it is simple enough to introduce the main threat model: offline guessing against a recorded transcript. Its weaknesses, including dependence on an idealized encryption model and symmetric password storage, motivate the need for later protocols.

SRP (1998) was selected because of its practical significance. It remains one of the most widely deployed PAKEs, appearing in systems such as 1Password and Apple HomeKit. It also clearly illustrates the shift from symmetric password storage to asymmetric verifier-based storage. The server stores a salt and verifier rather than the plaintext password, which reduces the impact of server compromise. At the same time, SRP’s verifier structure leaves it vulnerable to the pre-computation attack described in §5.1, making it a useful bridge between EKE and OPAQUE.

OPAQUE (2018) was selected as the modern strong aPAKE example. It formalizes the stronger security goal of resisting pre-computation, uses an OPRF as the central tool, and gives a UC-security proof for the resulting construction. It is also a good example of modern compositional design: rather than building a monolithic password protocol, OPAQUE combines an OPRF and an authenticated key exchange, with each component serving a separate purpose.

6.2 Comparison

Across these three protocols, each generation fixes an important weakness of its predecessor and introduces a technique that motivates the next step. Table 1 summarizes the comparison.

	EKE (1992)	SRP (1998)	OPAQUE (2018)
Design strategy	Password as enc. key	Password as mask	OPRF + AKE
Symmetry	Symmetric	Asymmetric	Strong asymmetric
Resists offline guessing	Partial	Yes	Yes
Resists pre-computation	No	No	Yes
Perfect forward secrecy	No	Yes (SRP-6)	Yes
Provable security	Heuristic	Heuristic	UC, ROM
Server-side password material	Password/equiv.	Salt + verifier	OPRF key + envelope
Modularity	No	No	Yes
Deployment status	Academic	1Password, HomeKit	IETF standard

Table 1: Side-by-side comparison of the three protocols.

Three points are worth emphasizing. First, no PAKE in the survey satisfies all desirable properties simultaneously; the best-performing protocols satisfy 11 of the 13 criteria listed in the survey [1, Table 9]. PAKE design is therefore a Pareto trade-off among security, usability, and efficiency, rather than a search for a single protocol that dominates in every dimension. Second, the field has moved from monolithic protocols toward modular constructions. EKE is a single integrated design, while OPAQUE separates password

hiding, credential recovery, and authenticated key exchange into distinct components. Third, deployment lags theory. SRP was published in 1998 and remains widely deployed, even though OPAQUE has offered a cleaner security story since 2018. This gap between cryptographic design and production adoption is itself an important practical problem.

6.3 Open Problems

The survey identifies several directions where PAKE research remains active.

Post-quantum PAKE. The three protocols discussed here rely on discrete-logarithm-based Diffie-Hellman structure, which would be broken by a sufficiently large quantum computer. Lattice-based PAKEs exist [11–13], but constructing a quantum-safe *strong* aPAKE with OPAQUE-like guarantees remains an open direction. The main difficulty is replacing the discrete-log-based OPRF with a post-quantum analog while preserving a clean proof.

Anonymous PAKE. OPAQUE still reveals an identifier during login, since the server must look up the user’s password file. Recent work on anonymous PAKE [15] asks whether authentication can be achieved without revealing the user’s identity to the server. The challenge is that password files are normally indexed by user, so anonymous lookup requires either oblivious database techniques or a different storage architecture.

Identity-based PAKE. Cremers et al. [16] introduced CHIP and CRISP, identity-binding PAKEs in which the password is cryptographically tied to both parties’ identities. The motivating gap is that strong aPAKEs like OPAQUE protect against server compromise in the asymmetric client-server setting but offer no analogous guarantee in the symmetric setting, where both parties hold the password directly (for example, two devices owned by the same user pairing). CRISP, the stronger of the two, achieves pre-computation resistance in this symmetric setting, a property previously confined to saPAKEs. The cost is that each party must maintain structured per-identity password material rather than a single low-entropy password, which complicates the human-typed-password scenarios that originally motivated PAKE.

The deployment gap. Password-over-TLS works with existing HTTPS clients because password handling is separate from the cryptographic protocol. PAKEs, in contrast, require client-side protocol support, such as an OPRF implementation and envelope decryption. Closing this gap is largely an engineering problem. Candidate approaches include browser-level OPAQUE support, integration with TLS exporter mechanisms, and WebAuthn-style registration flows.

7 Conclusion

PAKE has matured substantially since EKE. The field has moved from heuristic protocols vulnerable to subtle attacks toward constructions with clearer definitions, modular structure, and formal proofs. EKE introduced the basic idea that a weak password can authenticate a strong key exchange. SRP showed that verifier-based asymmetric storage could be practical. OPAQUE then strengthened the model by preventing useful pre-computation before server compromise, using an OPRF and an AKE in a clean compositional design. Looking ahead, the main pressure points are post-quantum security, stronger privacy during login, and the deployment friction that has kept older protocols in use despite weaker guarantees.

References

- [1] D. Wang, G. Li, and J. Jiang. Password-Authenticated Key Exchange Protocols: A Survey. *ACM Computing Surveys*, 58(6):Article 140, 2025.
- [2] S. Jarecki, H. Krawczyk, and J. Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. In *EUROCRYPT 2018*, pages 456–486, 2018.
- [3] D. Bourdrez, H. Krawczyk, K. Lewi, and C. A. Wood. The OPAQUE Augmented Password-Authenticated Key Exchange (aPAKE) Protocol. RFC 9807, IRTF/CFRG, 2025.
- [4] H. Krawczyk. HMQV: A high-performance secure Diffie-Hellman protocol. In *CRYPTO 2005*, pages 546–566, 2005.
- [5] H. Krawczyk and P. Eronen. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). RFC 5869, IETF, 2010.
- [6] S. Jarecki, A. Kiayias, and H. Krawczyk. Round-optimal password-protected secret sharing and T-PAKE in the password-only model. In *ASIACRYPT 2014*, pages 233–253, 2014.
- [7] M. Steiner, G. Tsudik, and M. Waidner. Refinement and extension of encrypted key exchange. *ACM SIGOPS Operating Systems Review*, 29(3):22–30, 1995.
- [8] M. Abdalla and D. Pointcheval. Simple password-based encrypted key exchange protocols. In *CT-RSA 2005*, pages 191–208, 2005.
- [9] D. Cash, E. Kiltz, and V. Shoup. The twin Diffie-Hellman problem and applications. In *EUROCRYPT 2008*, pages 127–145, 2008.
- [10] F. Hao and P. Y. A. Ryan. Password authenticated key exchange by juggling. In *SPW 2008*, pages 159–171, 2008.
- [11] J. Katz and V. Vaikuntanathan. Smooth projective hashing and password-based authenticated key exchange from lattices. In *ASIACRYPT 2009*, pages 636–652, 2009.
- [12] S. Jiang, G. Gong, J. He, K. Nguyen, and H. Wang. PAKEs: New framework, new techniques and more efficient lattice-based constructions in the standard model. In *PKC 2020*, pages 396–427, 2020.
- [13] Z. Li, D. Wang, and E. Morais. Quantum-safe round-optimal password authentication for mobile devices. *IEEE Transactions on Dependable and Secure Computing*, 19(3):1885–1899, 2022.
- [14] B. F. Dos Santos, Y. Gu, S. Jarecki, and H. Krawczyk. Asymmetric PAKE with low computation and communication. In *EUROCRYPT 2022*, pages 127–156, 2022.
- [15] M. I. G. Vasco, A. L. P. del Pozo, and C. Soriente. A key for John Doe: Modeling and designing anonymous password-authenticated key exchange protocols. *IEEE Transactions on Dependable and Secure Computing*, 18(3):1336–1353, 2021.
- [16] C. Cremers, M. Naor, S. Paz, and E. Ronen. CHIP and CRISP: Protecting all parties against compromise through identity-binding PAKEs. In *CRYPTO 2022*, pages 668–698, 2022.
- [17] S. M. Bellovin and M. Merritt. Encrypted key exchange: password-based protocols secure against dictionary attacks. In *Proceedings 1992 IEEE Computer Society Symposium on Research in Security and Privacy*, pages 72–84, 1992.
- [18] T. Wu. The Secure Remote Password Protocol. In *Proceedings of the Internet Society Network and Distributed System Security Symposium (NDSS)*, pages 97–111, 1998.